

CHAP 13(2): pandas 데이터 처리 분석

pandas의 함수

울릉도는 몇 월에 바람이 가장 강할까?

조건에 맞게 골라내기, 빠진 값을 찾고 메워보기

타이타닉호의 승객 데이터 분석

[참고도서]

- 파이썬 Express
- 따라하며 배우는 파이썬과 데이터과학
- 으뜸 데이터분석과 머신러닝

판다스의 함수

- 데이터 프레임 중에서 몇 개의 행만을 가져오고자 할 때 방법
 - 처음 n개 행을 얻을 때 사용.
 - `head(n)`
 - 데이터프레임_변수[:n]
 - 마지막 n개 행을 얻고 싶을 때 사용
 - `tail(n)`
- 파일의 열 마다 가지고 있는 데이터 개수를 알려줌
 - `count()`
 - 예) 데이터프레임 **weather**에 있는 열 이름과 각 열의 데이터 개수.

```
>>> weather.count()
일시                3684
평균기온(° C)       3684
최대 풍속(m/s)      3680
평균 풍속(m/s)      3678
month              3684
dtype: int64
```

판다스의 함수

- 데이터를 특정한 값에 기반하여 묶는 기능(그룹핑)
 - `groupby('열이름')`
 - 인자는 그룹으로 묶을 때에 사용될 열의 레이블.
 - `groupby('열이름').mean()`
 - 여기에 `mean()`을 적용하면, 해당 그룹의 데이터들이 가진 값의 평균을 구할 수 있다.

울릉도는 몇 월에 바람이 가장 강할까?

- 울릉도의 기상 데이터에 있는 일일 평균 풍속 데이터를 바탕으로 몇 월에 바람이 가장 강한지 분석해 보자.
 - 출처. 기상자료개방 포털 (<https://data.kma.go.kr/cmmn/main.do>)

A	B	C	D
일시	평균기온(°C)	최대 풍속(m/s)	평균 풍속(m/s)
2010-08-01	28.7	8.3	3.4
2010-08-02	25.2	8.7	3.8
2010-08-03	22.1	6.3	2.9
2010-08-04	25.3	6.6	4.2
2010-08-05	27.2	9.1	5.6
2010-08-06	26.8	9.8	8
2010-08-07	27.5	9.1	5
2010-08-08	26.6	5.9	4
2010-08-09	26.9	5.1	3.1
2010-08-10	25.6	10.2	5.5
2010-08-11	24.6	9.4	4.8
2010-08-12	23.7	8.7	2.6
2010-08-13	24.3	10.9	4.6

weather.csv

기간: 2010-08-01 ~ 2020-08-31
자료량: 3684행

울릉도는 몇 월에 바람이 가장 강할까?

- 판다스 사용으로 각 달의 평균 풍속을 구하기.
 - ‘일시’ 열에서, 달의 정보를 가져와 ‘month’ 열을 추가한다.
 - ‘month’ 열을 사용하여, 월 별로 분리해서 평균을 구한다.

```
>>> weather
```

‘month’ 열 추가

	일시	평균기온(° C)	최대 풍속(m/s)	평균 풍속(m/s)	month
0	2010-08-01	28.7	8.3	3.4	8
1	2010-08-02	25.2	8.7	3.8	8
2	2010-08-03	22.1	6.3	2.9	8
3	2010-08-04	25.3	6.6	4.2	8
4	2010-08-05	27.2	9.1	5.6	8
...	...	...	...	...	...
3679	2020-08-27	26.8	6.0	3.6	8
3680	2020-08-28	27.2	6.5	3.5	8
3681	2020-08-29	27.3	8.1	5.1	8
3682	2020-08-30	25.3	9.4	4.2	8
3683	2020-08-31	22.4	7.9	5.5	8

```
[3684 rows x 5 columns]
```

울릉도는 몇 월에 바람이 가장 강할까?

- 일시 열에서 **year, month, day** 값 가져오기
 - ‘일시’ 열에서, 월(mm) 값을 가져오기

```
>>> pd.DatetimeIndex(weather['일시']).month
Int64Index([8, 8, 8, 8, 8, 8, 8, 8, 8, 8,
            ...,
            8, 8, 8, 8, 8, 8, 8, 8, 8, 8],
            dtype='int64', name='일시', length=3684)
```

- ‘일시’ 열에서, 년도(yyyy) 값을 가져오기

```
>>> pd.DatetimeIndex(weather['일시']).year
Int64Index([2010, 2010, 2010, 2010, 2010, 2010, 2010, 2010, 2010, 2010,
            ...,
            2020, 2020, 2020, 2020, 2020, 2020, 2020, 2020, 2020, 2020],
            dtype='int64', name='일시', length=3684)
```

울릉도는 몇 월에 바람이 가장 강할까?

- ‘일시’ 열에서, 일(dd) 값을 가져오기

```
>>> pd.DatetimeIndex(weather['일시']).day  
Int64Index([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10,  
            ...  
            22, 23, 24, 25, 26, 27, 28, 29, 30, 31],  
            dtype='int64', name='일시', length=3684)
```

- ‘month’ 열을 추가.

```
>>> data = pd.DatetimeIndex(weather['일시']).month  
>>> weather['month'] = data
```

올릉도는 몇 월에 바람이 가장 강할까?

- `groupby()`에 'month' 열의 레이블을 지정하여, 이 열의 값으로 그룹을 묶는다.
- `mean()`을 사용해서 그룹별로 평균을 구한다.

```
>>> means = weather.groupby('month').mean()
```

```
>>> means
```

month	평균기온(° C)	최대 풍속(m/s)	평균 풍속(m/s)
1	1.598387	8.158065	3.757419
2	2.136396	8.225357	3.946786
3	6.250323	8.871935	4.390291
4	11.064667	9.305017	4.622483
5	16.564194	8.548710	4.219355
6	19.616667	6.945667	3.461000
7	23.328387	7.322581	3.877419
8	24.836364	6.961290	3.656891
9	20.323667	6.896333	3.661667
10	15.383871	7.766774	3.961613
11	9.889667	8.013333	3.930667
12	3.753548	8.045484	3.817097

울릉도는 몇 월에 바람이 가장 강할까?

#샘플소스 (pandas의 groupby()로 분류)

```
import pandas as pd
import matplotlib.pyplot as plt
import datetime as dt
```

#한글이 포함되어 있어 지정
#한글인코딩 방식을 지정

```
weather = pd.read_csv('F:/DATA/weather.csv', encoding='CP949')
```

```
weather['month'] = pd.DatetimeIndex(weather['일시']).month
```

#'일시' 열에서, 월 값을 가져와 'month' 열을 추가

```
means = weather.groupby('month').mean()
```

```
plt.plot(means['평균 풍속(m/s)'], 'blue')
```

```
plt.show()
```

```
##
```

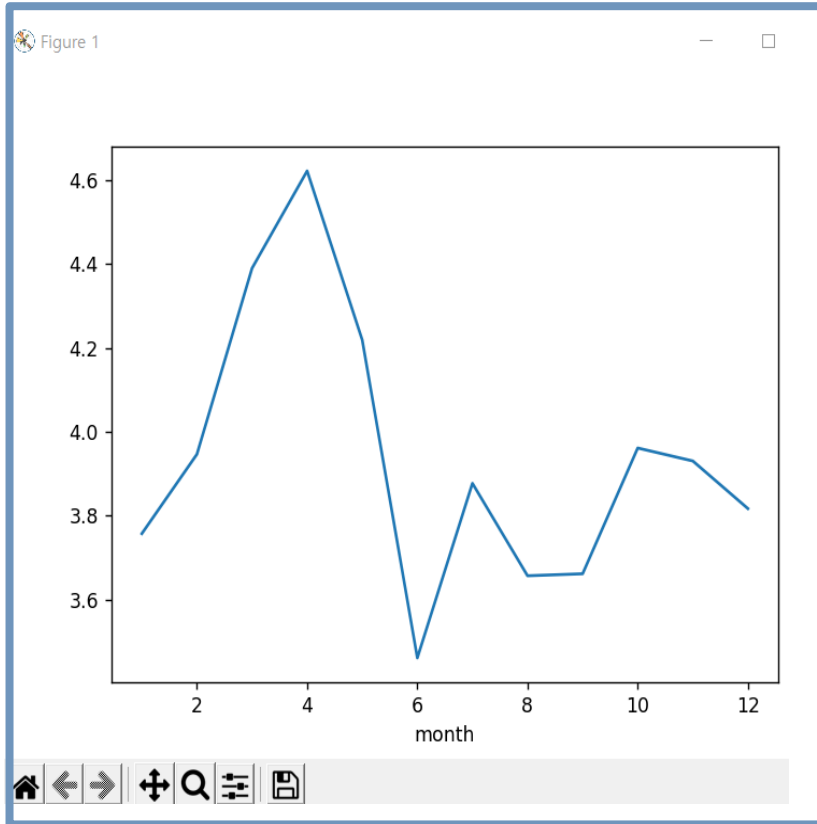
```
means['평균 풍속(m/s)'].plot(kind='bar')
```

```
plt.show()
```

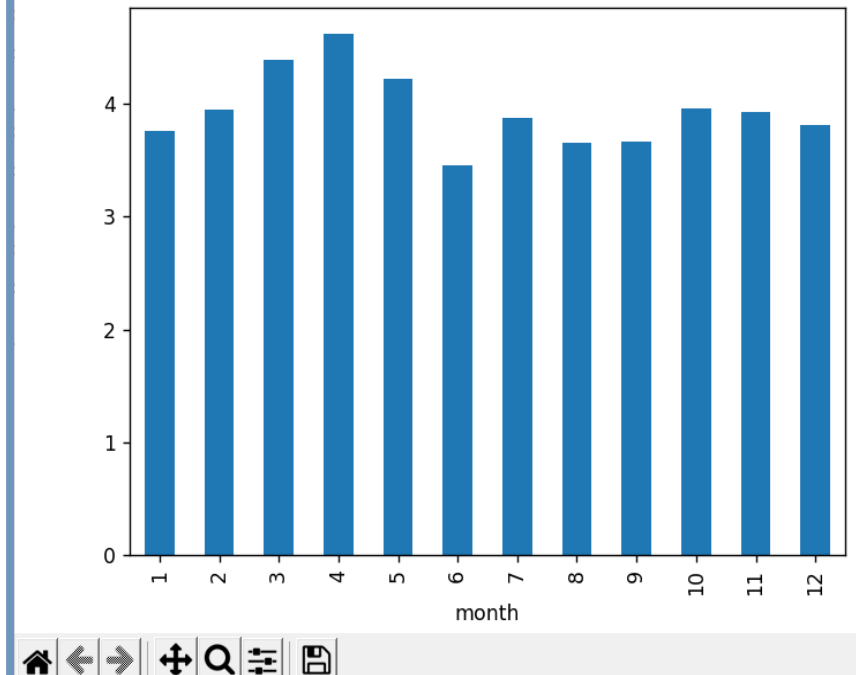
울릉도는 몇 월에 바람이 가장 강할까?

😊출력 화면

```
plt.plot(means['평균 풍속(m/s)'], 'blue')  
plt.show()
```



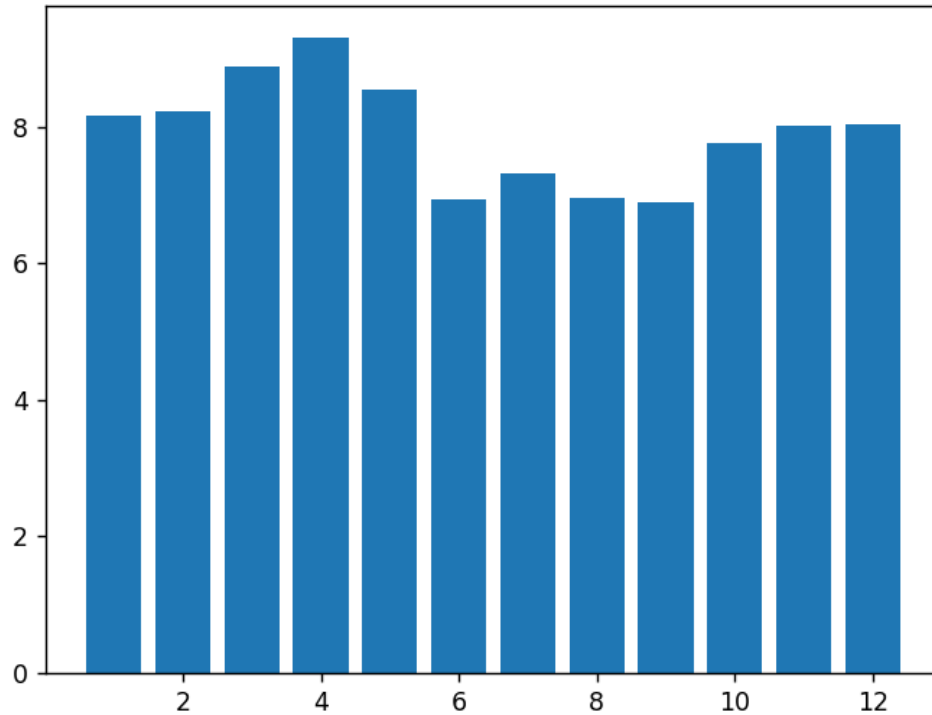
```
means['평균 풍속(m/s)'].plot(kind='bar')  
plt.show()
```



울릉도는 몇 월에 바람이 가장 강할까?

```
>>> plt.bar(means.index, means['평균 풍속(m/s)'])  
>>> plt.show()
```

Figure 1



울릉도는 몇 월에 바람이 가장 강할까?

- 각 달의 평균 풍속을 구하기.(반복문 사용)
 - ‘일시’ 열에서, 달의 정보를 가져와 ‘month’ 열을 추가한다.
 - ‘month’ 열을 사용하여, 월 별로 분리해서 평균을 구한다.

groupby()와
비교

‘month’ 열 추가

```
>>> weather
```

	일시	평균기온(° C)	최대 풍속(m/s)	평균 풍속(m/s)	month
0	2010-08-01	28.7	8.3	3.4	8
1	2010-08-02	25.2	8.7	3.8	8
2	2010-08-03	22.1	6.3	2.9	8
3	2010-08-04	25.3	6.6	4.2	8
4	2010-08-05	27.2	9.1	5.6	8
...	...	...	...	...	...
3679	2020-08-27	26.8	6.0	3.6	8
3680	2020-08-28	27.2	6.5	3.5	8
3681	2020-08-29	27.3	8.1	5.1	8
3682	2020-08-30	25.3	9.4	4.2	8
3683	2020-08-31	22.4	7.9	5.5	8

```
[3684 rows x 5 columns]
```

울릉도는 몇 월에 바람이 가장 강할까?

- 각 달의 평균 풍속을 구하기.(반복문 사용)
 - 'month' 열을 반복문으로 월 별로 분리해서 평균을 구한다.

groupby()와
비교

'month' 열의 값이 1인 행만 추출. (차례로 1~12까지 반복)

```
>>> weather[ weather[ 'month' ] == 1 ]
```

	일시	평균기온(° C)	최대 풍속(m/s)	평균 풍속(m/s)	month
153	2011-01-01	0.8	19.0	11.1	1
154	2011-01-02	0.1	8.5	6.0	1
155	2011-01-03	-0.3	10.2	6.5	1
156	2011-01-04	1.7	6.8	3.1	1
157	2011-01-05	1.2	6.9	4.1	1
...	...	...	...	...	...
3466	2020-01-27	5.4	15.3	7.6	1
3467	2020-01-28	5.7	14.9	11.6	1
3468	2020-01-29	4.1	9.0	6.6	1
3469	2020-01-30	1.8	10.2	7.7	1
3470	2020-01-31	1.9	10.2	5.3	1

```
[310 rows x 5 columns]
```

울릉도는 몇 월에 바람이 가장 강할까?

*groupby()와
비교*

'month' 열의 값이 1인 행을 추출하여 monthly[0]에 할당.
(차례로 1~12까지 반복)

```
>>> monthly[0]
```

	일시	평균기온(° C)	최대 풍속(m/s)	평균 풍속(m/s)	month
153	2011-01-01	0.8	19.0	11.1	1
154	2011-01-02	0.1	8.5	6.0	1
155	2011-01-03	-0.3	10.2	6.5	1
156	2011-01-04	1.7	6.8	3.1	1
157	2011-01-05	1.2	6.9	4.1	1
...	...	...	...	...	...
3466	2020-01-27	5.4	15.3	7.6	1
3467	2020-01-28	5.7	14.9	11.6	1
3468	2020-01-29	4.1	9.0	6.6	1
3469	2020-01-30	1.8	10.2	7.7	1
3470	2020-01-31	1.9	10.2	5.3	1

[310 rows x 5 columns]

울릉도는 몇 월에 바람이 가장 강할까?

groupby()와 비교 #샘플소스(pandas 사용)
import pandas as pd
import matplotlib.pyplot as plt
weather = pd.read_csv('F:/DATA/weather.csv', encoding='CP949')
monthly = [0 for x in range(12)] #달별로 구분된 12개의 데이터프레임
monthly_wind = [0 for x in range(12)] #각 달의 평균 풍속을 담은 리스트

마지막으로 해당 행의 데이터가 측정된 달을 기록한 열을 추가
weather['month'] = pd.DatetimeIndex(weather['일시']).month

for i in range(12) :
 monthly[i] = weather[weather['month'] == i + 1] # 월별로 분리
 monthly_wind[i] = monthly[i].mean()['평균 풍속(m/s)']
 #월별 평균 계산

print(monthly_wind)
plt.plot(range(1,13), monthly_wind, 'red')
plt.show()

울릉도는 몇 월에 바람이 가장 강할까?

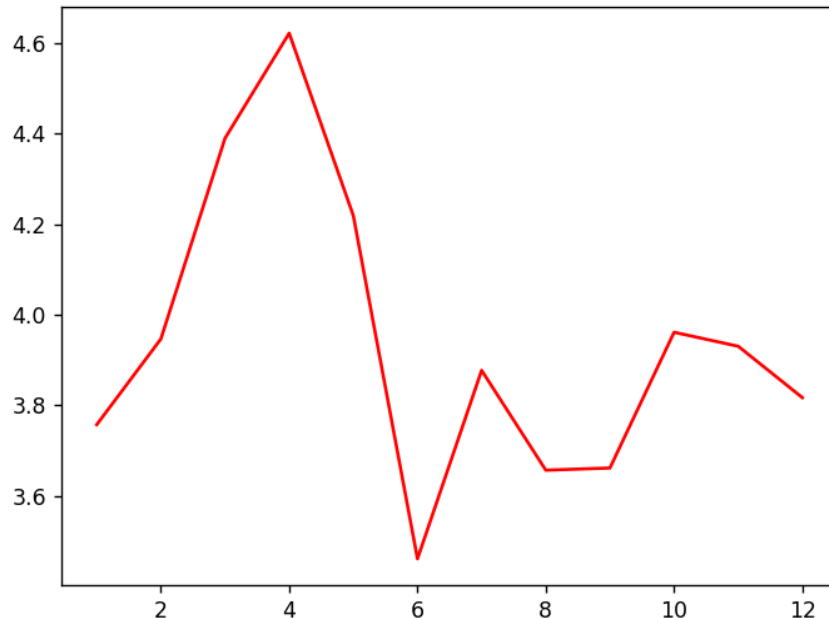
😊출력 화면

'month' 열의 값에 따라 분류하고 구한 월별 평균

```
[3.7574193548387096, 3.946785714285714,  
4.390291262135922, 4.62248322147651,  
4.219354838709678, 3.461,  
3.8774193548387097, 3.656891495601173,  
3.6616666666666666, 3.9616129032258067,  
3.9306666666666666, 3.817096774193548]
```

groupby()와
비교

- □



울릉도는 몇 월에 바람이 가장 강할까?

groupby()와
비교

- csv 모듈을 사용하여 각 달의 평균 풍속을 구하기.
 - 각 달별로 측정 풍속 누적 합계를 저장할 리스트를 생성.(원소 12개) 변수 `monthly_wind`
 - 각 달마다 측정 데이터가 존재하는 일수를 담은 리스트 생성.(원소 12개) 변수 `days_counted`
 - 각 행의 데이터를 읽어 **달 정보(문자열의 [5:7])**에 따라 해당 월의 풍속 정보에 이 데이터를 더하고, 일수에도 해당 월 정보를 증가.
 - 누적된 풍속 데이터를 계산된 일수로 나누어 월 평균 풍속을 구한다.

울릉도는 몇 월에 바람이 가장 강할까?

#샘플소스(csv모듈 사용)

```
import csv
import matplotlib.pyplot as plt
f = open('F:/DATA/weather.csv') # CSV 파일 열어 f에 저장
data = csv.reader(f)           # reader() 함수로 읽기
header = next(data)             # 헤더를 제거
monthly_wind = [ 0 for x in range(12) ] # 월별 풍속을 담을 리스트
days_counted = [ 0 for x in range(12) ] # 각 달마다 측정된 일수
for row in data:
    month = int(row[0][5:7])      # 0번 열에서 월 정보 추출
    if row[3] != "":             # 풍속 데이터 존재하는지 확인
        wind = float(row[3])     # 풍속을 얻어 온다.
        monthly_wind[month-1] += wind # 해당 월에 풍속 데이터 추가
        days_counted[month-1] += 1  # 해당 월의 일수를 증가
for i in range(12):
    monthly_wind[i] /= days_counted[i] # 일수로 나누어 월평균 구하기
print(monthly_wind)
plt.plot(monthly_wind, 'blue')
plt.show()
f.close() # 파일을 닫기
```

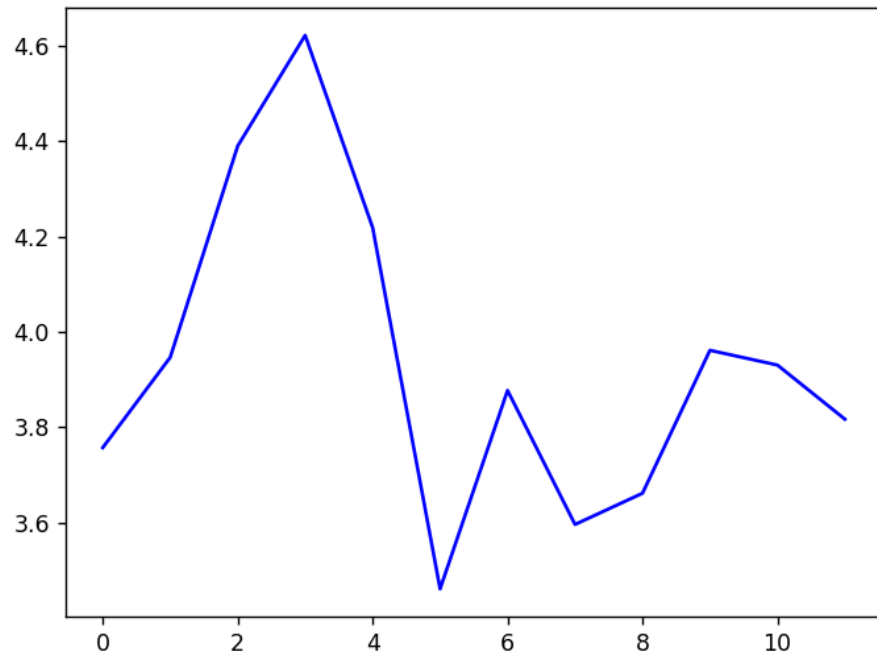
groupby()와
비교

울릉도는 몇 월에 바람이 가장 강할까?

😊출력 화면

```
[3.7574193548387074, 3.946785714285715,  
4.390291262135921, 4.62248322147651,  
4.219354838709678, 3.4609999999999985,  
3.8774193548387106, 3.5961290322580615,  
3.6616666666666666, 3.961612903225804,  
3.93066666666666654, 3.817096774193548]
```

groupby()와
비교



조건에 맞게 골라내기

- 조건에 맞게 골라내기(필터링)
- 예) 울릉도에 초속 10m/s 이상의 강풍이 불었던 날들 찾기.
 - ‘최대 풍속(m/s)’ 열의 값이 10.0 이상인지 확인하여 참이면 True, 아니면 False인 Boolean Series를 생성.

```
>>> weather[ '최대 풍속(m/s)' ] >=10.0
0          False
1          False
2          False
3          False
4          False
...
3679       False
3680       False
3681       False
3682       False
3683       False
Name: 최대 풍속(m/s), Length: 3684, dtype: bool
```

조건에 맞게 골라내기

- 불리언 인덱싱을 사용하여 추출.
 - Boolean Series를 인덱스로 사용하여, True인 index 행만 추출.

```
>>> weather[weather['최대 풍속(m/s)'] >=10.0]
```

	일시	평균기온(° C)	최대 풍속(m/s)	평균 풍속(m/s)	month
9	2010-08-10	25.6	10.2	5.5	8
12	2010-08-13	24.3	10.9	4.6	8
13	2010-08-14	25.0	10.8	4.4	8
14	2010-08-15	24.5	16.9	10.3	8
29	2010-08-30	26.2	10.5	6.2	8
...	...	...	...	...	...
3658	2020-08-06	23.0	13.0	5.3	8
3661	2020-08-09	23.4	11.0	6.2	8
3662	2020-08-10	25.1	13.0	5.7	8
3663	2020-08-11	24.5	15.1	8.7	8
3666	2020-08-14	28.3	12.0	9.3	8

[837 rows x 5 columns]

조건에 맞게 골라내기

- 예) 울릉도에 초속 15m/s 이상의 강풍이 불었던 날들 찾기.
 - `weather[ weather['최대 풍속(m/s)'] >=15.0 ]`

```
>>> weather[weather['최대 풍속(m/s)'] >=15.0]
```

	일시	평균기온(° C)	최대 풍속(m/s)	평균 풍속(m/s)	month
14	2010-08-15	24.5	16.9	10.3	8
113	2010-11-22	11.0	16.6	8.8	11
153	2011-01-01	0.8	19.0	11.1	1
195	2011-02-12	-2.9	17.6	9.7	2
257	2011-04-15	15.9	18.8	8.8	4
...	...	...	...	...	...
3435	2019-12-27	1.8	16.4	6.5	12
3447	2020-01-08	6.5	22.2	8.2	1
3466	2020-01-27	5.4	15.3	7.6	1
3622	2020-07-01	16.8	19.7	8.7	7
3663	2020-08-11	24.5	15.1	8.7	8

[74 rows x 5 columns]

조건에 맞게 골라내기

- 예) 울릉도에 초속 20m/s 이상의 강풍이 불었던 날들 찾기.
 - `weather[ weather['최대 풍속(m/s)'] >=20.0 ]`

```
>>> weather[weather['최대 풍속(m/s)'] >=20.0]
```

	일시	평균기온(° C)	최대 풍속(m/s)	평균 풍속(m/s)	month
261	2011-04-19	4.4	21.6	12.1	4
1850	2015-08-25	20.1	25.3	14.9	8
1851	2015-08-26	17.4	22.6	8.1	8
2988	2018-10-06	19.4	26.0	7.0	10
3339	2019-09-22	15.7	23.1	11.9	9
3340	2019-09-23	15.0	25.8	11.0	9
3447	2020-01-08	6.5	22.2	8.2	1

빠진 값을 찾고 메워보기

- 판다스에서는 결손값을 NaN으로 나타낸다(혹은 NA로 표기함).
판다스는 결손값missing data을 탐지하고 수정하는 함수를 제공.
 - 결손값이란 데이터가 아예 수집되지 않았거나, 측정 장치의 고장 등으로 데이터가 없는 경우이다. 이 값을 탐지하고 수정해 보자.
 - 변수[열이름].isna()
 - 예) 데이터프레임 변수 weather의 열 '최대 풍속(m/s)'에서 탐지

```
>>> weather[weather['최대 풍속(m/s)'].isna()]
```

	일시	평균기온(° C)	최대 풍속(m/s)	평균 풍속(m/s)	month
559	2012-02-11	-0.7	NaN	NaN	2
560	2012-02-12	0.4	NaN	NaN	2
561	2012-02-13	4.0	NaN	NaN	2
3183	2019-04-19	7.8	NaN	2.3	4

- 빠진 데이터를 메우기.
 - 결손값을 새로운 값으로 채우기.
 - fillna(채울값, inplace=True)
결손값을 지정된 값으로 채운다. 이 작업이 원본에 반영되도록 설정.

```
>>> weather.fillna(0, inplace=True)
```


빠진 값을 찾고 메워보기

- 최대풍속의 결손값이 있던 2012년 02월11일의 데이터를 출력해 보자. 결손값 NaN이 아니라 0이 채워진 것을 확인할 수 있다.

```
>>> weather[weather['일시']=='2012-02-11']
```

	일시	평균기온(°C)	최대 풍속(m/s)	평균 풍속(m/s)	month
559	2012-02-11	-0.7	0.0	0.0	2

- 데이터 생성에 결손값 표기
 - np.nan #결손값을 의미

```
>>> data = pd.Series([1,2,np.nan,4])
>>>
>>> data
0    1.0
1    2.0
2    NaN
3    4.0
dtype: float64
>>>
```

타이타닉호의 승객 데이터 분석

- 타이타닉호의 승객 파일에서 여러 가지 정보를 추출해본다
 - 타이타닉 승객정보 파일.

titanic.csv 자료량: 891행

	A	B	C	D	E	F	G	H	I	J	K	L
1	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
2	1	0	3	Braund, M	male	22	1	0	A/5 21171	7.25		S
3	2	1	1	Cumings, I	female	38	1	0	PC 17599	71.2833	C85	C
4	3	1	3	Heikkinen,	female	26	0	0	STON/O2.	7.925		S
5	4	1	1	Futrelle, M	female	35	1	0	113803	53.1	C123	S
6	5	0	3	Allen, Mr.	male	35	0	0	373450	8.05		S
7	6	0	3	Moran, Mr	male		0	0	330877	8.4583		Q
8	7	0	1	McCarthy,	male	54	0	0	17463	51.8625	E46	S
9	8	0	3	Palsson, M	male	2	3	1	349909	21.075		S
10	9	1	3	Johnson, N	female	27	0	2	347742	11.1333		S

타이타닉호의 승객 데이터 분석

- 데이터 필드
 - PassengerId: 승객의 ID이다.
 - Survived: 생존 여부
 - Pclass: 탑승 등급을 나타낸다. 클래스 1, 클래스 2, 클래스 3의 3가지 클래스가 있다.
 - Name: 승객의 이름.
 - Sex: 승객의 성별.
 - Age: 승객의 나이.
 - SibSp: 승객에게 형제 자매와 배우자가 있음을 나타낸다.
 - Parch: 승객이 혼자인지 또는 가족이 있는지 여부.
 - Ticket: 승객의 티켓 번호.
 - Fare: 운임.
 - Cabin : 승객의 선실.
 - Embarked: 탑승한 지역

타이타닉호의 승객 데이터 분석

- titanic.csv 파일 읽기

```
import pandas as pd
titanic = pd.read_csv('titanic.csv')
```

😊 실행 화면

```
>>> titanic
```

	PassengerId	Survived	Pclass	...	Fare	Cabin	Embarked
0	1	0	3	...	7.2500	NaN	S
1	2	1	1	...	71.2833	C85	C
2	3	1	3	...	7.9250	NaN	S
3	4	1	1	...	53.1000	C123	S
4	5	0	3	...	8.0500	NaN	S
..	...	...	...	...	...	...	...
886	887	0	2	...	13.0000	NaN	S
887	888	1	1	...	30.0000	B42	S
888	889	0	3	...	23.4500	NaN	S
889	890	1	1	...	30.0000	C148	C
890	891	0	3	...	7.7500	NaN	Q

```
[891 rows x 12 columns]
```

타이타닉호의 승객 데이터 분석

- titanic.csv 파일에 대한 데이터에 대한 간략한 개요를 제공

```
import pandas as pd  
titanic = pd.read_csv('titanic.csv')
```

😊실행 화면

```
>>> titanic.describe()
```

	PassengerId	Survived	Pclass	...	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	...	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	...	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	...	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	...	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	...	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	...	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	...	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	...	8.000000	6.000000	512.329200

```
[8 rows x 7 columns]
```

```
>>>
```

타이타닉호의 승객 데이터 분석

- titanic.csv 파일에서 20세 미만의 승객?

☺실행 화면

```
>>> titanic[titanic["Age"] < 20]
```

	PassengerId	Survived	Pclass	...	Fare	Cabin	Embarked
7	8	0	3	...	21.0750	NaN	S
9	10	1	2	...	30.0708	NaN	C
10	11	1	3	...	16.7000	G6	S
14	15	0	3	...	7.8542	NaN	S
16	17	0	3	...	29.1250	NaN	Q
..	...	...	...	...	...	...	...
855	856	1	3	...	9.3500	NaN	S
869	870	1	3	...	11.1333	NaN	S
875	876	1	3	...	7.2250	NaN	C
877	878	0	3	...	7.8958	NaN	S
887	888	1	1	...	30.0000	B42	S

```
[164 rows x 12 columns]>>>
```

타이타닉호의 승객 데이터 분석

- titanic.csv 파일에서 20세 미만의 승객의 이름?

😊 실행 화면

```
>>> titanic.loc[titanic['Age'] < 20, 'Name']
7          Palsson, Master. Gosta Leonard
9      Nasser, Mrs. Nicholas (Adele Achem)
10         Sandstrom, Miss. Marguerite Rut
14  Vestrom, Miss. Hulda Amanda Adolfina
16              Rice, Master. Eugene
...
855          Aks, Mrs. Sam (Leah Rosen)
869      Johnson, Master. Harold Theodor
875      Najib, Miss. Adele Kiamie "Jane"
877              Petroff, Mr. Nedelio
887          Graham, Miss. Margaret Edith
Name: Name, Length: 164, dtype: object
>>>
```

타이타닉호의 승객 데이터 분석

- titanic.csv 파일에서
 - 승객의 평균 나이?
 - 승객의 생존율 평균?
 - 성별 평균 생존율?
 - 20세 미만 승객의 생존율?

-titanic[['Sex', 'Survived']]로 두 열을 선택
-groupby('Sex') 메소드로 성별로 그룹구성
-각 성별의 생존율을 계산

☺실행 화면

```
>>> titanic["Age"].mean()
29.69911764705882
>>> titanic['Survived'].mean()
0.3838383838383838
>>> titanic[["Sex", "Survived"]].groupby("Sex").mean()
Survived
Sex
female 0.742038
male   0.188908
>>> titanic.loc[titanic['Age'] < 20, 'Survived'].mean()
0.4817073170731707
>>>
```


타이타닉호의 승객 데이터 분석

- titanic.csv 파일에서
 - 클래스별(Pclass) 생존율의 평균값 구하기.

😊실행 화면

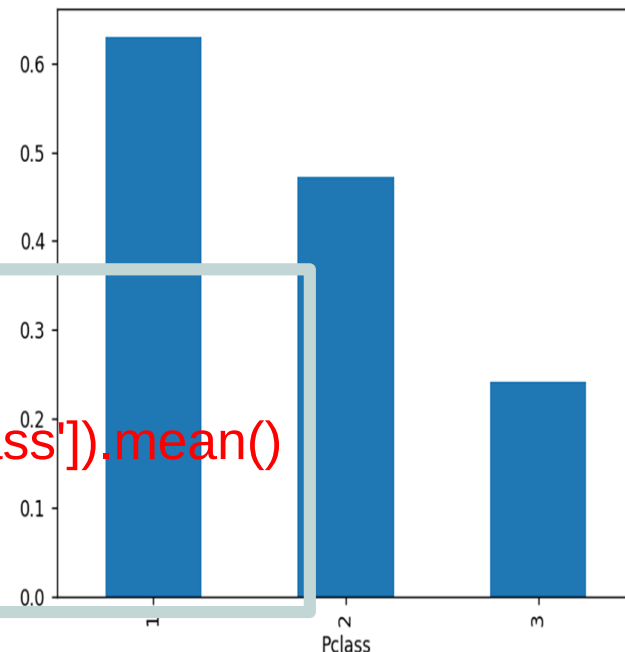
```
>>> titanic[['Survived','Pclass']].groupby(['Pclass']).mean()
```

Survived

Pclass

1	0.629630
2	0.472826
3	0.242363

```
import pandas as pd
titanic = pd.read_csv('titanic.csv')
p= titanic[['Survived','Pclass']].groupby(['Pclass']).mean()
p['Survived'].plot(kind='bar')
plt.show()
```



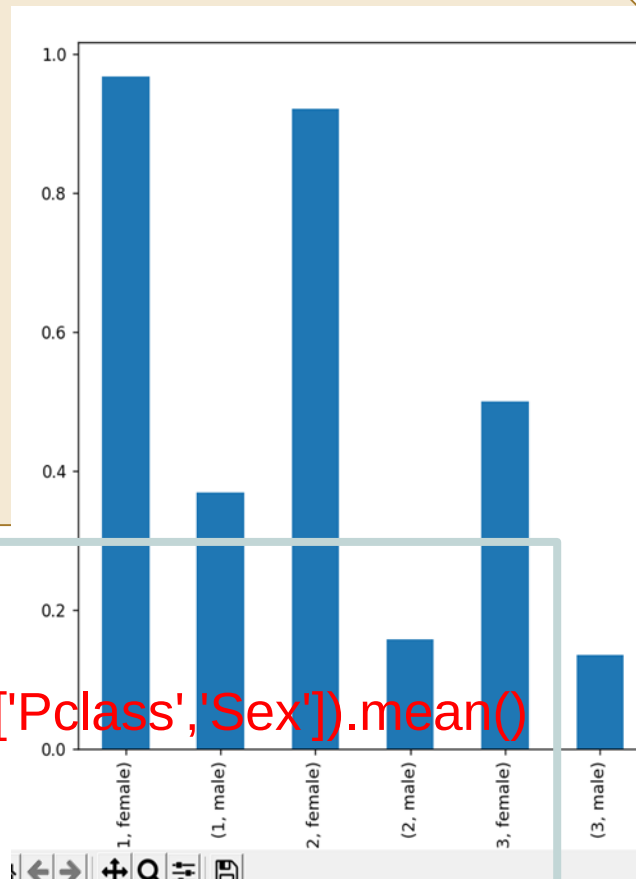
타이타닉호의 승객 데이터 분석

- titanic.csv 파일에서
 - 클래스와 성별(Pclass, Sex열) 생존율의 평균값 구하기.

😊실행 화면

```
>>> titanic[['Survived','Pclass','Sex']].groupby(['Pclass','Sex']).mean()
```

Pclass	Sex	Survived
1	female	0.968085
	male	0.368852
2	female	0.921053
	male	0.157407
3	female	0.500000
	male	0.135447

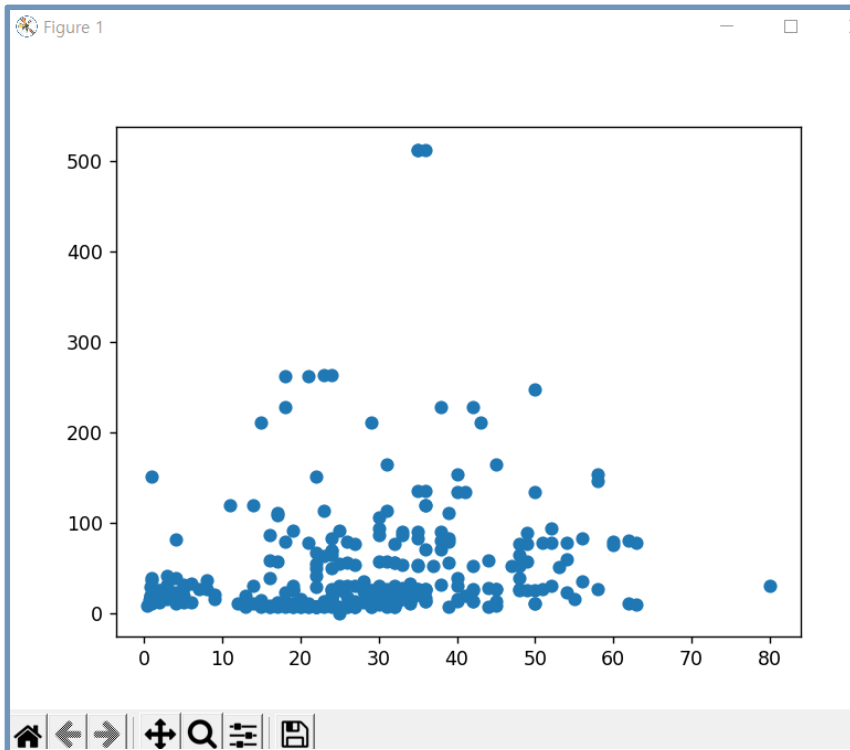


```
import pandas as pd
titanic = pd.read_csv('titanic.csv')
p= titanic[['Survived','Pclass','Sex']].groupby(['Pclass','Sex']).mean()
p['Survived'].plot(kind='bar')
plt.show()
```

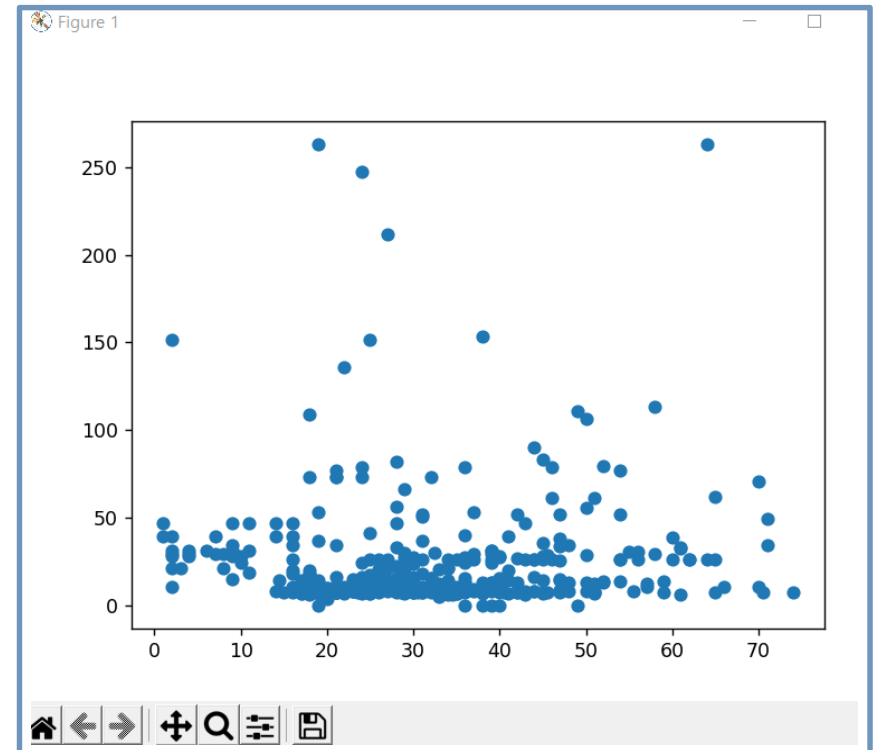
타이타닉호의 승객 데이터 분석

- 나이와 요금 사이의 관계를 알아보자
 - 생존자의 분포와 사망자의 분포간의 차이가 있는가 살펴보자.

생존자 분포



사망자 분포



타이타닉호의 승객 데이터 분석

😊 실행 화면

```
>>> p1=titanic[titanic['Survived']==1] #생존
>>> plt.scatter(x=p1.Age,y=p1.Fare)
>>> plt.show()

>>> p2=titanic[titanic['Survived']==0] #사망
>>> plt.scatter(x=p2.Age,y=p2.Fare)
>>> plt.show()
```